

CMPSC 202: Counting Steps Examples

1. Iteration

Algorithm CalculateSum(A, n)

Input: An array A of n numbers

Output: number sum

```
1: sum  $\leftarrow$  0
2: for  $i \leftarrow 1$  to  $n$  do
3:   sum  $\leftarrow$  sum +  $A[i]$ 
4: end for
```

For this algorithm, $T(n) = 3n + 1$. Inside the loop, there are 3 steps: getting the element $A[i]$, adding it to the sum, and updating the sum variable. The loop runs n times, contributing $3n$ steps. The extra 1 accounts for the initialization of the sum variable before the loop. Moreover, $T(n)$ is linear in n , i.e., $T(n)$ is $\mathcal{O}(n)$.

2. Nested Loops

Algorithm PrintAllPairs(A, n)

Input: An array A of n numbers

```
1: for  $i \leftarrow 1$  to  $n$  do
2:   for  $j \leftarrow 1$  to  $n$  do
3:     print  $A[i], A[j]$ 
4:   end for
5: end for
```

This algorithm iterates from 1 to n for both the outer and inner loops, printing all possible pairs of elements in the array A . The total number of steps is given by $T(n) = 2 \cdot n \cdot n = 2n^2$, where each iteration of the inner loop counts as two steps. Moreover, $T(n)$ is quadratic in n , i.e., $T(n)$ is $\mathcal{O}(n^2)$.

3. Conditional Logic

Algorithm CountEvens(A, n)

Input: An array A of n integers

Output: number count

```
1: count  $\leftarrow$  0
2: for  $i \leftarrow 1$  to  $n$  do
3:   if  $A[i] \bmod 2 == 0$  then
4:     count  $\leftarrow$  count + 1
5:   end if
6: end for
```

This algorithm counts the number of even integers in the array A . In the best case, where there are no even numbers, the inner step of incrementing the count never executes. In the worst case, where all numbers are even, the count increment executes every iteration, leading to the maximum number of steps. Line 4 contributes two steps. If all numbers are even, this code executes n times, resulting in a total of $2n$ steps for the loop. Then the total worst case number of steps is $T(n) = 2n + 2$, where the extra 2 accounts for the initialization of the count variable. Moreover, $T(n)$ is linear in n , i.e., $T(n)$ is $\mathcal{O}(n)$.

4. Non-Linear Increment (Logarithmic)

Algorithm MysteryLoop(n)

Input: An integer $n > 1$

Output: The final value of count

```
1: count  $\leftarrow$  0
2:  $i \leftarrow 1$ 
3: while  $i < n$  do
4:   count  $\leftarrow$  count + 1
5:    $i \leftarrow i \times 2$ 
6: end while
```

This algorithm counts the number of times the loop executes, with the loop variable doubling each iteration. The loop starts with $i = 1$ and continues while $i < n$, doubling i each time. Therefore, the number of iterations is approximately $\log_2 n$. Exactly, the number of iterations is $\lceil \log_2 n \rceil$. Each iteration has 2 steps: incrementing the count and doubling i . Additionally, there are 2 steps outside the loop for initializing count and i . Hence, the total number of steps is $T(n) = 2\lceil \log_2 n \rceil + 2$, which is logarithmic in n , i.e., $T(n)$ is $\mathcal{O}(\log n)$.